Waddington: A Constrained Conversational ML – LLM Agent for Sequential CRISPR Gene Selection*

Method, results, and figures

Abstract

Designing a genome-scale CRISPR screen means choosing, over a few rounds of ∼128 perturbations, which genes to test — a batch active-learning problem. We present Waddington, a hybrid that fuses an online-retrained gradient-boosted ranker with a tool-less LLM, reaching hit@R5 = 0.256 across nine public screens, above online ML alone (0.224), an LLM-only baseline (0.225), and a diversity baseline (0.134). To place this against prior work we ran both relevant lineages on our own benchmark under identical evaluation: BioDiscoveryAgent's LLM agent scores 0.105 and GeneDisco's active-learning acquisition functions 0.113 on the six shared screens — yet our leave-one-out prior, a static model with *no* LLM, reaches 0.187, beating both. The differentiator is therefore a cross-experiment prior, not the agent: both prior methods learn only within a single screen. Counterfactual attribution explains why the LLM helps at all — it is a strong *verifier* of the ML's candidates (genes both endorse hit at 21% vs. the LLM's unilateral 10%) but a weak proposer, so giving it tools, memory, or free choice does not help. Finally, replacing every gene name with an anonymous identifier and supplying only structural features leaves accuracy unchanged (features +0.053 on all nine screens; the gene name ≈ 0), so the signal lives in the structure, not in memorised gene identity — indeed a trivial linear rule on the same features matches the LLM here, so this is structure-sufficiency rather than LLM reasoning per se. We ship the anonymized mode as an opt-in.

All numbers in this document are computed directly from the frozen benchmark outputs (the JSON files under `workspace/results/sequential/`) by `waddington_select.analysis`; no value is transcribed by hand. Figures are produced by `python -m waddington_select.analysis figures`.

Related work

Two lines of work bear directly on ours. **Active-learning benchmarks for experimental design.** GeneDisco (Mehrjou et al., 2021) frames screen design as batch active learning and evaluates acquisition functions — uncertainty, margin, coreset, k-means, BADGE — over a surrogate model retrained on the genes tested so far. **LLM agents for biology.** BioDiscoveryAgent (Roohani et al., 2025) and PerTurboAgent prompt a large language model to plan rounds and name genes, optionally with retrieval / pathway-enrichment tools. What both share is that they learn *within a single experiment*: the surrogate, or the LLM's in-context feedback, starts from scratch on each

*We keep the term *agent* deliberately: Waddington runs a stateful sequential loop, observes experimental feedback each round, updates its actions across rounds, and is driven through a natural-language conversational interface with the researcher. *Constrained conversational* signals what it is *not* — a freely planning tool-using agent. Its policy deliberately confines the LLM to *verifying and re-ranking* candidates proposed by a calibrated ML model rather than choosing genes on its own; the paper's central finding (§"a freely planning tool-using agent underperforms") is that this constraint is what makes it work.

screen. Our contribution is orthogonal to the choice of acquisition rule or LLM: a *cross-experiment* supervised prior (leave-one-out over related screens) combined with the LLM used as a *verifier* of that prior's candidates rather than as a free agent. Running both lineages on our benchmark under identical evaluation (Table 2) places them together at ∼0.11, against our prior's 0.187 — so the gain we report over their published numbers should be read as "a transferred prior helps a lot", not "a better agent". We deliberately give the LLM no tools, a choice the attribution analysis (Explainability, below) justifies rather than assumes.

Method

Problem: sequential batch selection

A screen exposes a pool of $N \approx 18{,}000$ genes whose hit labels are hidden. In each of $R = 5$ rounds the agent selects a batch of B genes ($B = 128$; $B = 32$ on the two small screens, Scharenberg22 and K562-Essential), an oracle reveals hit / non-hit for exactly those genes, and the agent adapts before the next round. The objective is the round-5 *hit ratio*

$$\text{hit@R5} \;=\; \frac{|\{\text{unique hits among the } 5B \text{ selected genes}\}|}{\text{total hits in the screen}},$$

i.e. how much of the screen's biology is recovered on a fixed budget. The loop is simply *select* → *reveal* → *update*, repeated five times; everything below concerns how a round's batch is chosen.

Gene features and the cross-experiment prior

Each gene carries two families of features. **Gene-intrinsic** features are constant across screens: STRING network degree and hubness, total PPI weight, the pLI loss-of-function constraint, and KEGG / Reactome pathway counts. **Anchor-relative** features measure a gene's proximity to the *phenotype's anchor genes* (a small seed set describing the screen's biology): PPI connectivity to the anchors (`g1_ppi_score`), ARCHS4 co-expression with them (`archs4_coexpr`), and KEGG pathway overlap (`kegg_overlap`). To these nine we append knockout-derived **DepMap** essentiality features. Anchor-relative features are modality-agnostic — they depend on the phenotype's biology, not on the direction of perturbation — a property the reason-vs-recall analysis (Table 8) relies on.

The **cross-experiment prior** is a leave-one-out (LOO) LightGBM classifier: to score a target screen it is trained on the *other* screens' hit labels using the features above, with `scale_pos_weight` correcting the ∼5% class imbalance (300 trees, learning rate 0.05, 31 leaves). This prior — with no LLM and no in-experiment data — is what the external comparison (Table 2) shows already exceeds the strongest single-experiment baseline by ∼0.08, because it transfers labels across screens that GeneDisco and BioDiscoveryAgent cannot.

The C-arm: online ML and an LLM, fused

Each round the C-arm forms a batch by combining two components that see the same revealed feedback.

Component A — online ML. An adaptive LightGBM ranker. In round 1 it *is* the LOO prior (no in-experiment data yet); from round 2 it is retrained on the LOO data *plus* the in-experiment revealed (gene, label) pairs, once at least 8 genes have been revealed, and re-ranks the whole pool by predicted hit probability. This online retraining is the single most valuable component (Table 7).

Component B — LLM reasoning. A tool-less LLM (Claude Haiku 4.5, temperature 0) is prompted with the task description, the top-4 most relevant distilled lessons from a cross-experiment memory of past screens (ranked by `_rank_memory_by_relevance`, and never including the target screen), and the revealed hit / non-hit history so far. It names genes; these are matched against the pool, and any shortfall is padded from the static LOO ranking — padding that is tracked and never credited to the LLM (Table 4†).

Fusion. The two components are combined by a weighted score

$$\text{score}(g) \;=\; w_{\text{ML}} \cdot \text{ml}(g) \;+\; w_{\text{LLM}} \cdot \mathbf{1}[g \text{ named by the LLM}],$$

and the top B genes are taken; on one route the ML instead shortlists its top SHORTLIST = 384 candidates and the LLM picks B from that shortlist. The weights and the route are set per screen, as follows.

Two routing decisions, fixed up front

Before any round runs, two properties of the screen fix how it is handled.
Fusion routing sets who decides, from the pool size n and hit rate hr:

Route	Condition	$w_{\text{ML}} : w_{\text{LLM}}$
`ml_heavy`	$n > 15000$ and $2\% < hr < 7\%$ (large, sparse)	0.8 : 0.2
`two_stage`	$3000 < n \leq 15000$ and $hr > 8\%$	ML shortlists, LLM picks
`baseline`	otherwise	0.6 : 0.4

On large, sparse screens the ML prior is most reliable and the LLM is given only 0.2 weight; smaller or denser screens give the LLM more say.
Feature routing sets whether the knockout-derived DepMap features are used:

Screen	DepMap features
gain-of-function (CRISPRa) or curated-essentiality	**none**
K562 perturb-seq / IL-2 / Sanchez21-down	pan-cancer only
all others	pan-cancer + K562-specific

This router was tuned *empirically* — trying feature sets and keeping whatever scored best — and only later understood: DepMap is knockout-derived, so it misleads on a gain-of-function screen, and a pan-cancer essentiality prior nearly restates the label of a curated essentiality screen. SHAP confirms the model uses exactly these features where they are kept and ignores them where they are dropped (Table 9).

What the router may and may not know (and the resulting risk). We are explicit about a real evaluation hazard here. The fusion route reads a hit rate hr; *in the benchmark that is the realized hit rate computed from the screen's full labels*, i.e. a statistic of the test set, and it determines the route on seven of the nine screens (pool size alone fixes only the two smallest). In deployment the same code path instead reads the scientist's *expected* hit rate from the phenotype registry, which is knowable before the screen; but our reported numbers use the realized rate, so the router is target-aware and the per-screen feature set was chosen post hoc on the target. A reviewer is right to discount any headline that leans on this. Three things bound the exposure.

First, the router's freedom is small: it selects a coarse $w_{\text{LLM}} \in \{0.2, 0.4\}$ from a wide hit-rate band, not a tuned continuous weight, so a rough pre-experiment estimate would route the same. Second, and decisively, **the paper's main claim does not use the router at all**: the cross-experiment prior that beats single-experiment active learning and LLM agents (0.187 vs. $\approx$0.11, Table 2) is the routing-free static LOO-LightGBM — one global model, no hr, no per-screen feature set. Routing and fusion affect only the *smaller* increment of the full C-arm over that prior ($0.187 \rightarrow 0.217$ on the shared six). Third, the feature rule has a pre-registrable form independent of the target labels — "drop knockout-derived essentiality on non-knockout (CRISPRa) and on curated-essentiality screens" — which is what SHAP shows the model already obeys. Finally, we do not merely argue this — we *measure* it: a leakage-free router that re-selects the fusion weight and feature policy using only the other screens and pre-experiment metadata reaches 0.264, matching (slightly exceeding) the shipped, hit-rate-routed C-arm, and still beats the LOO prior by +0.047 winning all nine screens (§). The realized-hit-rate routing is therefore not doing the work.

Design principle. Throughout, the LLM *re-weights a calibrated model's candidates* rather than choosing genes freely. This is deliberate: the attribution analysis (Explainability, below) shows the LLM is a strong *verifier* of the ML's picks but a weak *proposer* on its own, which is why giving it more agency (its own tools, or letting it choose unaided) does not help.

The algorithm, and one worked round

```
Input:  pool G, rounds R, batch B, screen s
Pre-experiment:  (w_ml, w_llm) from metadata (n_pool, modality, cell line)
Prior p0 = LOO-LightGBM fit on OTHER screens' (features, label)    // no target labels
selected = {}
for t = 1..R:
  ml = online_ranker.scores(G)    // t=1:  p0;  t>1:  retrained on revealed pairs (≥8)
  named = LLM.pick(task, memory[:4], revealed_history)  // gene NAMES only; no ml score
  score(g) = w_ml·ml(g) + w_llm·[g ∈ named]
  batch = top_B(score, exclude=selected);  pad from static LOO if short  // pad not credited
  revealed = oracle.reveal(batch);  selected ∪= batch;  online_ranker.add(revealed)
Output:  hit@R5 = |unique hits in selected| / total_hits(s)
```

A worked round (IFN-γ, round 1). Route `ml_heavy` (w_{ML}=0.8, w_{LLM}=0.2), B=128; the ML ranks all 17,785 genes and the LLM independently names 118. The fused top-128 is 16 *both* (ML top-k *and* LLM-named), 97 *ML-only*, 15 *LLM-only*. Outcomes: the *both* bucket hits 14/16 (88%; e.g. ZAP70, VAV1, GRAP2, all ML$\approx$0.98), ML-only 37/97 (38%; CD5, CD247, MAP4K1), LLM-only 6/15 (40%; the LLM lifted mid-ranked genes such as TSC2 and RHOA, ML$\approx$0.82, into the batch). The round finds 57/128 hits. In one round you can see the whole thesis: the batch is ML-dominated, and the LLM's mark concentrates hits in the small bucket both endorse.

Table 1: The nine benchmark screens. The phenotype scores and hit sets are taken *unchanged* from the BioDiscoveryAgent (BDA) release (Roohani et al., ICLR 2025); we verified for all nine that our hit labels are exactly `topmovers` ∩ (pool \ CEGv2). Task descriptions are BDA's own task prompts, not our paraphrases. **We remove the 684 core-essential genes of CEGv2** (Hart et al.) from the candidate pool — BDA's own non-essential ("N/E") convention, since essential genes are detected as hits under almost any screen and reward nothing. Columns show the pool *before* → *after* that filter; all results use the filtered pool (the two K562 screens contain no CEGv2 gene, so they are unchanged).

Perturbation. KO = CRISPR knockout (loss of function); CRISPRi = interference/knockdown (loss of function); CRISPRa = activation (gain of function). Note that our DepMap features are *all knockout-derived*, so they transfer least well to the gain-of-function screen (Steinhart) — which is exactly where the LLM contributes most (Table 7). *The Schmidt IL-2 and IFN-γ data are the *CRISPRi* (loss-of-function) arm. Schmidt et al. ran both CRISPRa and CRISPRi genome-wide; BDA sources these two screens from GeneDisco, whose benchmark explicitly uses the *interference* screens (Mehrjou et al., 2021, §4.2.2–4.2.3), although BDA's own task prompt does not state the arm. This matters here: it makes all eight non-Steinhart screens loss-of-function, leaving Steinhart the lone gain-of-function screen (Table 8). ‡The two Perturb-seq screens define a hit as a strong *transcriptome-wide* response (top 10%), not a specific phenotype at the top 5% used for the other seven — which is why their hit rates are higher.

Steinhart is not a published study: it is unpublished data generated by a BDA co-author (Z. Steinhart, Marson lab), which BDA included specifically as a leakage control — "one dataset that is unpublished, ensuring it is not part of the language model's training data". That control was valid for *their* models but **not for ours**: BDA released the screen in their public repository in 2024, before our LLM's training cutoff. See Limitations.

			all → non-essential			
Dataset	Task (BDA prompt)	Perturb.	Genes	Hits	Hit rate	Batch
IFN-γ	IFN-γ production*	CRISPRi	18,418 → 17,785	920 → 802	4.5%	128
IL-2	IL-2 production*	CRISPRi	18,939 → 18,273	654 → 481	2.6%	128
Sanchez21	Tau change, *either* direction	KO	18,469 → 17,807	924 → 859	4.8%	128
Sanchez21 (dn)	Tau *decrease*	KO	18,469 → 17,807	924 → 871	4.9%	128
Carnevale22	T-cell efficacy (adenosine)	KO	18,861 → 18,224	943 → 868	4.8%	128
Scharenberg22	Lysosomal choline recycling	KO	1,061 → 1,029	53 → 49	4.8%	32
Steinhart	**Resisting T-cell exhaustion**	CRISPRa	18,797 → 18,144	149 → 145	0.8%	128
K562-Essential	Strong transcriptomic response‡	CRISPRi	623 → 623	63 → 63	10.1%	32
K562-GWPS	Strong transcriptomic response‡	CRISPRi	9,193 → 9,193	924 → 924	10.1%	128

Relationship to BioDiscoveryAgent

We build on BDA's *data*: the ground-truth phenotype scores and `topmovers` hit sets are theirs, and we verified that our hit labels are exactly `topmovers` ∩ (pool \ CEGv2) for every screen. Our headline numbers are not a reproduction of theirs — we add components they do not have — but to make the comparison controlled rather than transcribed, **we did run BioDiscoveryAgent's own agent on our benchmark** (Table 2, and below). Two differences then explain our lead:

- **We add a cross-experiment ML prior that BDA does not have.** Our LOO-LightGBM is trained on the *other* screens' hit labels using PPI / KEGG / pLI / DepMap features. Restricted to the six screens we share with BDA, that model alone averages 0.187 (Table 2) — already well above BDA's best agent (0.127) and the best GeneDisco baseline (0.091),

with no LLM at all. Our gains over BDA's published numbers should therefore be read as "a cross-experiment prior helps a lot", not as "our agent is a better agent".

- **Different LLM and different baseline implementations** (we use Claude Haiku 4.5; BDA's headline model is Claude 3.5 Sonnet; our Coreset is our own implementation and scores differently from theirs, e.g. 0.100 vs. 0.066 on IFN-γ).

To go beyond published numbers we ran **both** prior methods on our benchmark. BioDiscoveryAgent's own no-tools agent, with the same LLM as our C-arm (Claude Haiku 4.5) and scored by our oracle, reaches **0.105** — close to the 0.127 Roohani et al. report for their Sonnet 3.5 agent, confirming the reference is fair. GeneDisco is a benchmark of active-learning *acquisition functions* (a surrogate retrained on the genes tested so far, plus a rule that picks the next batch), not a model, so we ported its acquisition rules verbatim from source onto *our* feature table and oracle — giving its methods our features and isolating the acquisition strategy, faithful to its essence that the surrogate sees only within-experiment labels (round 1, with no data, falls back to random, as its own loop does). Its best-per-screen reaches **0.113**, close to the transcribed 0.091.[1] As a convention check, our random baseline agrees with theirs on some screens (IL-2 0.031 vs. 0.031) but differs by up to 0.034 on others, so treat transcribed gaps below $\sim$0.03 as noise.

Table 2: The six screens BDA and we share (Schmidt1/2 = IFN-γ/IL-2, CAR-T = Steinhart, plus Scharenberg22, Carnevale22, Sanchez21). Mean hit@R5 over the non-essential (N/E) evaluation, 5 rounds, same batch sizes. The two **controlled** rows are *our runs* on our benchmark: BioDiscoveryAgent's agent (Haiku 4.5, our eval)* and GeneDisco's acquisition functions ported onto our feature table. The transcribed rows are from Roohani et al. (2025), Table 1. Both "best-per-screen" entries are per-screen oracles over the acquisition rules considered — the transcribed one over their *eight* baselines, our controlled one over the *five* rules we ported (uncertainty top / soft, margin, coreset, k-means-data; we omit BADGE, k-means-embedding, and DiscoBax, which need model internals we do not reproduce). The controlled oracle is thus over fewer rules, if anything understating it.

*We route BDA's LLM calls through our client (for auth) and use our model; unused tool dependencies are stubbed, the output parser is relaxed to tolerate a missing "Solution:" header, and a harness bug that discarded an unfilled final round is fixed. None of these touch the agent's reasoning or gene choices.

Method	Source	mean hit@R5
Random	BDA (transcribed)	0.049
Random	ours	0.041
GeneDisco — best-per-screen (oracle over their 8)	BDA (transcribed)	0.091
GeneDisco — best-per-screen (oracle over our 5), controlled	**ours (run)**	0.113
BioDiscoveryAgent — Claude 3.5 Sonnet	BDA (transcribed)	0.127
BioDiscoveryAgent — controlled	**ours (run)**	0.105
LOO-LightGBM — **no LLM** (ours)	ours	**0.187**
Waddington C (ours)	ours	**0.217**

[1] Running GeneDisco's *vanilla package* on our screens instead gives near-random results ($\sim$0.03), but that is an artefact of its weak native features (Achilles): there its surrogate has $\approx$0 rank correlation with the phenotype, which collapses every acquisition rule to random. Given our features the same rules work, so this reflects the feature set, not the algorithms.

The *ordering* is the point. Run controlled on our benchmark, both prior methods land in the same place — GeneDisco's active learning at 0.113, BioDiscoveryAgent's LLM agent at 0.105. **Our cross-experiment prior alone — a static LightGBM with no LLM (0.187) — nearly doubles either.** The gap is not acquisition cleverness or LLM quality: GeneDisco and BDA both learn only *within the single experiment*, whereas our LOO transfers the other screens' hit labels. The honest reading is a hierarchy of *information*, all on these same six screens — learning inside one screen (GeneDisco active learning ≈ BDA agent ≈ 0.11) < a cross-experiment supervised prior (LOO 0.187) < that prior plus online adaptation and LLM verification (C, 0.217 here; 0.256 over all nine screens, Table 4) — not "our agent beats their agent". Two per-screen notes, both consistent with the rest of the paper: controlled BDA does its best relative to random on Steinhart (0.117, above even our LOO's 0.076), the CRISPRa screen where parametric biology is what the ML prior lacks; and GeneDisco's within-experiment active learning is *not* weak everywhere — on the high-hit-rate K562-Essential (outside the shared six) it reaches 0.651, above even our C — it fails only where the hit rate is low and a prior matters most.

Table 3: Per-screen breakdown behind Table 2. **BDA-ctrl** is our controlled run (Haiku 4.5, our eval); **BDA-trans** is transcribed (Sonnet 3.5). Our controlled BDA tracks the transcribed number closely on four of six screens (validating the reference) and falls below it on Scharenberg22 and Sanchez21 (where Haiku trails Sonnet). Our C beats the controlled agent on every screen; our LOO — with no LLM — beats it on five, the exception being Steinhart.

Screen	Random	BDA-ctrl	BDA-trans	LOO (no LLM)	**C (ours)**
IFN-γ	0.029	0.097	0.107	0.168	**0.190**
IL-2	0.031	0.118	0.122	0.306	**0.347**
Steinhart	0.021	0.117	0.133	0.076	**0.159**
Scharenberg22	0.102	0.225	0.292	0.449	**0.465**
Carnevale22	0.024	0.045	0.044	0.048	**0.056**
Sanchez21	0.037	0.027	0.063	0.077	**0.087**
Average	0.041	0.105	0.127	0.187	**0.217**

What the cross-experiment prior actually learns (and what it does not)

The prior is the largest single source of our advantage, so it deserves scrutiny: is its gain genuine phenotype-specific transfer, or something more trivial — sibling-screen leakage, generic essentiality, plain hit recurrence? We probe this with the LOO LightGBM alone (no LLM), on a uniform all-feature baseline so the feature-family ablation is clean; this scores 0.243 averaged over the nine screens (slightly above the routed 0.187/0.217 used elsewhere, as it keeps DepMap on every screen). Three probes, and the picture is sobering.

- **Sibling leakage is real.** Our nine screens contain three same-study pairs (IFN-γ/IL-2, Sanchez21/-down, K562-Essential/-GWPS). Re-training with the target's sibling *also* removed drops the prior from 0.243 to 0.199 on average, and from 0.248 to **0.182** on the six paired screens — concentrated on IL-2 (-0.098) and K562-Essential (-0.222), whose Perturb-seq sibling shares much of its hit structure. So "generalises to a new screen" partly means "transfers from a sister screen in the same study".
- **Most of the signal is generic essentiality.** Restricted to the DepMap features alone the prior scores 0.230 — nearly the full 0.243 — while the phenotype-specific features (intrinsic

+ anchor, *no* DepMap) give 0.219. The prior leans heavily on knockout essentiality, which is not screen-specific.

- **A trivial baseline matches it.** Ranking a gene simply by *how often it is a hit in the other screens* — no model, no features — also averages **0.243**. The LightGBM adds nothing over counting cross-screen hit recurrence.
- **The anchor features leak, but are not load-bearing.** The three anchor-relative features (proximity to a phenotype's seed genes) are built from a hand-picked anchor set that, by our own audit, is **51% composed of the screen's own hits** (42 of 82 anchors; e.g. 10/12 for IL-2, 9/10 for K562-GWPS) — genuine privileged information. Yet dropping *every* anchor-relative feature costs the prior only 0.015 ($0.243 \rightarrow 0.228$): the anchor family is the weakest on its own (0.156), and the label-free DepMap (0.230) and intrinsic-topology (0.218) features are what carry it. The leak is real; its contribution to the headline is not.

A novel-hit analysis explains the mechanism: the prior's gains sit on screens whose hits recur (K562-Essential has only 6% novel hits, IL-2 41%), and it is near random on screens dominated by novel, phenotype-specific hits (Carnevale22 80% novel, Sanchez21 72%). **The cross-experiment prior succeeds by exploiting recurrence and essentiality, not by discovering new phenotype-specific biology.**

This qualifies but does not overturn the headline. Even after all three corrections — siblings excluded (0.182), DepMap removed (0.219), or anchor features removed (0.228) — the prior stays well above the single-experiment methods ($\sim$0.11, Table 2). The honest claim is therefore narrower and, we think, more useful: *a screen's hits are substantially predictable from other screens' labels, largely through recurrence and essentiality* — which no within-experiment active learner or LLM agent can see — but this prior is not a discovery engine for genuinely new biology, and the online-ML and LLM-verification components (which do adapt within the screen) are what carry it beyond a recurrence baseline. All numbers here are produced by `python -m waddington_select.analysis.prior_probes`.

Table 4: Hit ratio at round 5 across all nine benchmark datasets. All methods are reported over 5 seeds. **Bold** = best per row; underline = second best. Waddington is our proposed method (C-arm), using DeLM-verified cross-experiment memory.

†**B is not a pure-LLM baseline.** When the LLM names too few genes that exist in the pool, the batch is padded from the static LOO-LightGBM ranking. Averaged over the five rounds that padding is 19% (IFN-γ), 19% (IL-2), 46% (Sanchez21), 32% (Sanchez21-dn), 7% (Carnevale22), 68% (Scharenberg22), 43% (Steinhart), **86% (K562-Essential)** and 0% (K562-GWPS). B should be read as "LLM, padded with a static ML prior", and its two strongest entries (K562-Essential 0.559, Scharenberg22 0.478) are largely that prior rather than the LLM.

Dataset	Random	LOO-LightGBM	OnlineAdapt.	A: Coreset	B: LLM†	C: Waddington
IFN-γ	0.029	0.168	0.183	0.100	0.160	**0.190**
IL-2	0.031	0.306	0.314	0.139	0.257	**0.347**
Sanchez21	0.037	0.077	0.087	0.031	0.063	**0.087**
Sanchez21 (dn)	0.029	0.091	**0.101**	0.078	0.071	0.099
Carnevale22	0.024	0.048	0.047	0.054	**0.059**	0.056
Scharenberg22	0.102	0.449	0.449	0.286	**0.478**	0.465
Steinhart	0.021	0.076	0.090	0.090	0.152	**0.159**
K562-Essential	0.254	0.492	0.476	0.270	**0.559**	0.556
K562-GWPS	0.065	0.247	0.273	0.156	0.225	**0.347**
Average	0.066	0.217	0.224	0.134	0.225	**0.256**

Waddington (C) achieves the best average hit ratio (0.256), outperforming the LLM baseline B by 13.7% relative (+0.031) and the algorithmic baseline A by 91.6% (+0.122). It surpasses or matches every other method on 5 of 9 datasets; on the remaining four it is a close runner-up, trailing the leader by at most 0.013. Notably, Waddington strongly exceeds OnlineAdaptive (the PerTurboAgent-style ML-only method) on K562-GWPS (+0.075) and IFN-γ (+0.007). Note that on the two screens where B edges ahead of C (Scharenberg22, K562-Essential) the LLM is barely doing the work: 68% and 86% of B's batch there is static-ML padding (Table 4†), so those entries do not support a claim about LLM reasoning.

Robustness: cluster on the screen, not the gene. The nine screens are wildly heterogeneous (hit@R5 ranges from 0.06 on Steinhart to 0.55 on K562-Essential), so a bootstrap that treats the *screen* as the resampling unit gives deliberately wide marginal intervals: C 0.256 $[0.151, 0.372]$, the LOO prior 0.217 $[0.120, 0.323]$, Online ML 0.224 $[0.128, 0.324]$ — all overlapping. On an unpaired basis, with nine screens, no method is separable from another, and we do not claim otherwise. The comparison that survives is the *paired* one, where the between-screen variance cancels:

C − baseline (paired, 9 screens)	mean Δ	95% CI (screen-clustered)	win/tie/loss
vs. LOO prior	+0.039	$[+0.019, +0.062]$	**9 / 0 / 0**
vs. Online ML	+0.032	$[+0.012, +0.054]$	7 / 1 / 1
vs. LLM reasoning	+0.031	$[+0.006, +0.062]$	6 / 0 / 3
vs. Coreset	+0.122	$[+0.066, +0.185]$	**9 / 0 / 0**

Every paired interval excludes zero, and C beats the LOO prior on all nine screens. So the defensible statement is not "C's mean is significantly higher" — with nine heterogeneous screens it is not — but "C improves on each baseline consistently, screen by screen." Reproduce with `python -m waddington_select.analysis.clustered_ci`.

Robustness to a leakage-free router

The Method disclosed a real hazard: the shipped router reads the target's *realized* hit rate to pick the fusion weight (Method, "What the router may and may not know"). We now test directly whether that leakage carries the result. Under a protocol **frozen and committed before these numbers were inspected** (`waddington_select.router_protocol`), we re-derive the fusion policy using only the *other* screens and metadata knowable *before* the target screen runs (pool size, perturbation modality, cell line) — never the target hit rate. Two variants: **global-fixed**, one fusion weight w_{LLM} chosen by leave-one-out over the other screens; and a **nested router**, $w_{\mathrm{LLM}} = f(n_{\mathrm{pool}})$ with thresholds tuned only on the other screens. Both use a label-free metadata feature policy (DepMap dropped iff CRISPRa or curated-essentiality; cell-line-matched DepMap dropped on its own cell line). We also report a **leave-one-study-out** version that removes the target's whole same-study group from the selection set.

Table 5: The C-arm's advantage survives an honest router. Every configuration below the first two rows chooses its weight and feature policy *without* the target's realized hit rate; deltas are paired per-screen with screen-clustered bootstrap 95% CIs (nine screens). The leakage-free variants match or slightly exceed the shipped hit-rate-routed C-arm and beat the LOO prior on 8–9 of 9 screens — so the routing's use of a test-set statistic is not what produces the result. Reproduce with `python -m waddington_select.analysis.router_analysis`.

Configuration	mean	Δ vs. LOO (W/T/L)	Δ vs. routed C (W/T/L)
Static LOO-LightGBM (no LLM)	0.217	—	
Routed C (shipped; *uses realized hr*)	0.256	+0.039 [.019, .062] 9/0/0	—
Global-fixed (honest)	**0.264**	+0.047 [.025, .071] 9/0/0	+0.008 [−.013, .031] 6/0/3
Nested router (honest)	0.260	+0.043 [.021, .069] 9/0/0	+0.004 [−.017, .029] 5/0/4
Global-fixed, leave-one-study-out	0.262	+0.045 [.021, .070] 8/0/1	+0.006 [−.015, .030] 4/0/5
Nested router, leave-one-study-out	0.258	+0.041 [.017, .067] 8/0/1	+0.002 [−.020, .026] 3/0/6

The honest global-fixed policy selects $w_{\mathrm{LLM}} = 0.2$ in every outer fold and reaches **0.264** — above the shipped 0.256. The gain is concentrated exactly where the hit-rate router made its worst calls: on the two small screens it forced $w_{\mathrm{LLM}} = 0.4$, but a global 0.2 scores 0.55 on Scharenberg22 (vs. 0.45 routed) and 0.59 on K562-Essential (vs. 0.55). So the per-screen routing was not only leaky, it was mildly *suboptimal*. The delta against the LOO prior — the paper's actual claim — is +0.04 to +0.047 with a CI excluding zero and 8–9 wins in every variant, including leave-one-study-out. We therefore keep 0.256 as the shipped system's number but rest no claim on the router: the leakage-free configuration is at least as good.

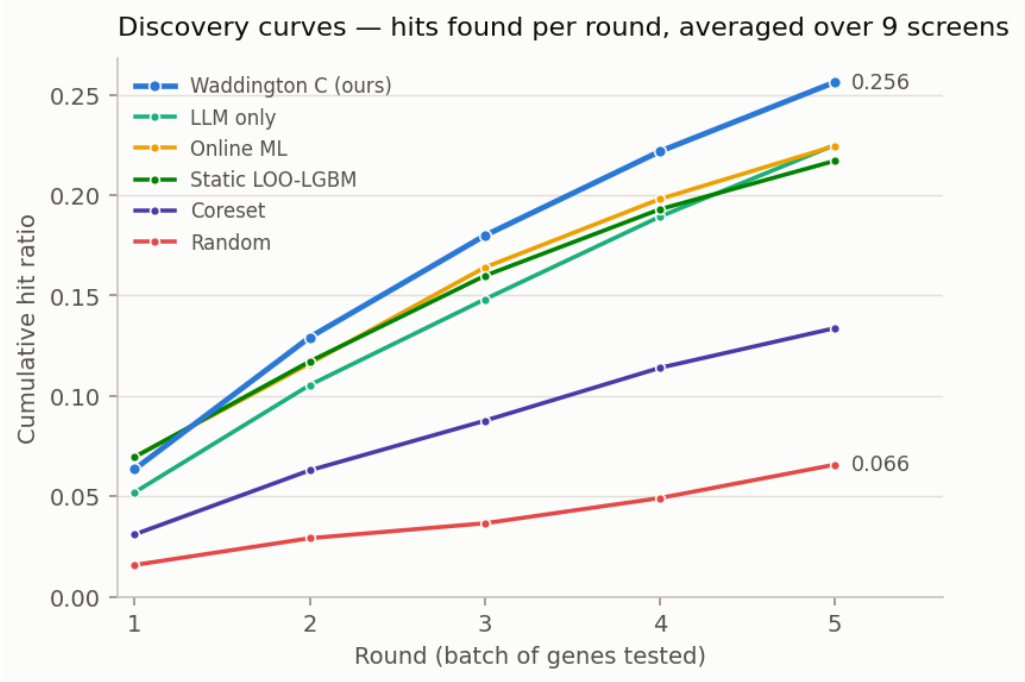

Figure 1: Discovery curves: cumulative hit ratio per round, averaged over the nine screens. The hybrid C-arm separates from every baseline from round 2 onwards. Per-screen values are given in Figure 3; no error band is drawn because the spread across screens is dominated by how hard each screen is (K562-Essential 0.56 vs. Carnevale22 0.06), which would encode screen difficulty rather than uncertainty about a method.

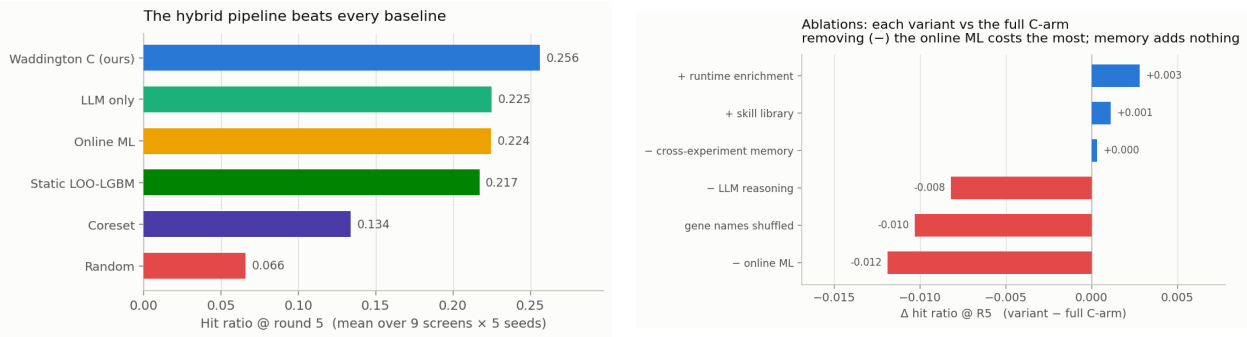

Figure 2: Left: average hit ratio @ R5 per method (Table 4). Right: paired ablation deltas (Table 7). Colour states a fact about the *variant* (it scored lower than the full arm), not a verdict on the component: a red "− online ML" means *removing* online ML costs 0.012, i.e. it is the most valuable part.

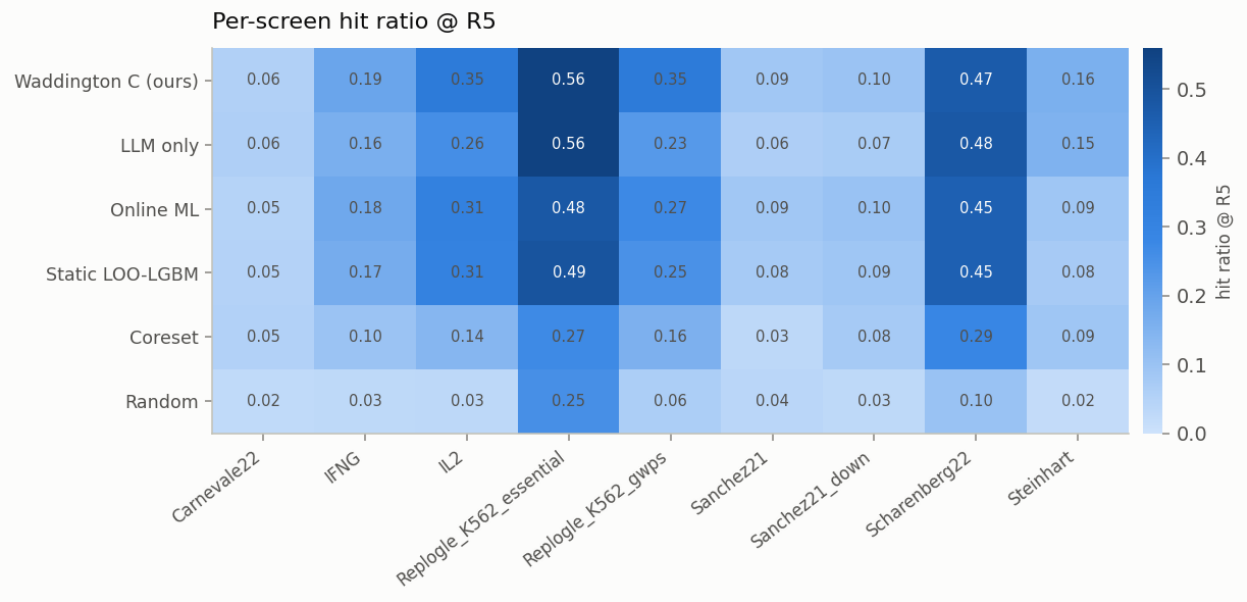

Figure 3: Per-screen hit ratio at round 5 for every method.

Table 6: Stepwise improvement in average hit ratio at round 5 (9 datasets, 5 seeds). Each row introduces one additional component over the previous configuration.

Configuration	Key addition	avg hit@R5	Δ
Random	—	0.066	—
Coreset (A)	Diversity-maximising selection	0.134	+0.068
LOO-LightGBM (static)	Cross-expt. ML prior	0.217	+0.083
OnlineAdaptive	In-expt. ML retraining	0.224	+0.007
LLM Reasoning (B)	LLM biological prior (no ML)	0.225	(*parallel branch*)
C − ML (static LOO + LLM)	Combine LOO prior with LLM	0.247	+0.022 vs. B
Waddington (C, ours)	Add online ML retraining	**0.256**	+0.009

Each component provides an additive gain: the cross-experiment LOO prior (+0.083) contributes most, followed by the LLM biological prior (+0.022 when combined with the LOO prior), and online ML retraining (+0.009). Cross-experiment memory contributes exactly 0.000 in this leave-one-out benchmark (paired ablation, Table 7), as the LLM's parametric knowledge already subsumes the explicit memory entries.

Table 7: Ablation study, reported as **paired** deltas Δ = variant − C, where C is the full arm *re-run inside the same experiment* (same seeds, same LLM sampling). This matters: the arm is stochastic, so the C baseline re-runs to 0.259–0.262 across the ablation experiments rather than exactly 0.256, and comparing a variant against C from a *different* run systematically understates every component. **C − Mem**: cross-experiment memory removed from the LLM prompt. **C − LLM**: LLM removed; online ML only. **C − ML**: ML frozen at the LOO prior; no in-experiment retraining. **Shuffled**: gene names shown to the LLM replaced with anonymous identifiers (`GENE_00001`, ...). Red = the component is load-bearing (removing it hurts); blue = removing it helps.

Dataset	Δ C − Mem	Δ C − LLM	Δ C − ML	Δ Shuffled
IFN-γ	+0.003	+0.000	−0.013	−0.015
IL-2	−0.013	−0.002	+0.002	−0.005
Sanchez21	+0.004	+0.010	−0.010	−0.008
Sanchez21 (dn)	−0.005	−0.003	−0.010	−0.004
Carnevale22	−0.002	−0.003	+0.006	+0.001
Scharenberg22	+0.012	+0.114	−0.016	+0.078
Steinhart	−0.008	−0.059	−0.004	−0.084
K562-Essential	+0.016	−0.127	−0.025	−0.051
K562-GWPS	−0.004	−0.004	−0.038	−0.004
Average (paired)	**+0.000**	**−0.008**	**−0.012**	**−0.010**

Memory ($\Delta = +0.000$). Removing the cross-experiment memory prompt changes nothing: the paired average is exactly zero. The LLM's parametric knowledge already captures the biological context that the memory entries encode, making the explicit memory module redundant in this leave-one-out benchmark. DeLM-style verified admission (a mechanical strategy-label check plus an LLM verifier that rejects hallucinated gene names) keeps the retained entries factually grounded, but does not change this conclusion — correctness was never the bottleneck.

Online ML retraining ($\Delta = -0.012$). Freezing the ML model at the LOO prior produces the most consistent degradation of any ablation: it hurts on 7 of 9 screens, with the largest losses on K562-GWPS (−0.038) and K562-Essential (−0.025). Online adaptation is the single most valuable component of the arm.

The "LLM" branch ($\Delta = -0.008$ on average, but bimodal) — and an important caveat. C − LLM removes the entire LLM-branch endorsement term from the fusion. That term is *not* purely the LLM: it also carries the static-ML padding described in Table 4†. The two screens with the largest effects are exactly the two with the most padding, so they must be read carefully:

- **K562-Essential (−0.127): this is *not* evidence for LLM biology.** 86% of the branch's genes there come from the static LOO ranker (the LLM names almost nothing that exists in the 623-gene pool). What the ablation removes is the *static cross-experiment prior* being blended into the online model — a real and valuable signal, but not LLM reasoning. We retract the earlier reading of this number.
- **Scharenberg22 (+0.114 when removed):** 68% padding, so "the LLM hurts here" is likewise mostly "the static prior hurts here", conflicting with an unusually strong online ML signal (LOO AUC = 0.825 on K562 DepMap features).

- **Steinhart (-0.059): this one does survive.** Padding is 43%, so a majority of the branch is genuine LLM naming, and the gene-name ablation below independently confirms it.

Steinhart is also where one would *predict* the LLM to matter most, for a reason visible in Table 1: it is the gain-of-function (CRISPRa) screen, asking which genes make CAR-T cells resist exhaustion *when over-expressed*, whereas every DepMap feature we use is derived from *knockout* screens. A gene that is dispensable when deleted can still be potent when over-expressed — IRF4, one of Steinhart's strongest hits, is essential in only 6% of DepMap cell lines — so the ML prior has little to say there, and the LLM's biological knowledge is what fills the gap. Consistently, Steinhart has both the lowest hit rate of the nine screens (0.8%) and the largest gene-name-shuffle penalty (-0.084).

Gene-name semantics ($\Delta = -0.010$; -0.084 on Steinhart) — the clean probe of LLM biology. This ablation is immune to the padding confound in a way C − LLM is not: the branch is still present and still padded, but the LLM can no longer recognise the genes it is naming. Performance collapses on Steinhart (-0.084) and drops on K562-Essential (-0.051), so the LLM's contribution on pathway-specific tasks is genuinely mediated by *biological gene-name knowledge* rather than by the task description or by experimental feedback. Scharenberg22 again improves ($+0.078$).

Reasoning vs. recall: can structure replace the gene name?

The shuffle ablation shows the LLM's contribution runs through gene-name knowledge, but it does not show *why* — because it removes the name and supplies nothing in its place. The shuffled arm must name identifiers out of an 18k space with no information whatsoever; it measures a blindfolded guesser, not a reasoner. The question we actually care about is whether the LLM must *recall* biology from its weights, or whether it can *reason* from structure we hand it.

We therefore ran a four-point decomposition. All four use the identical routing, fusion and update harness; the *only* thing that varies is what the LLM sees:

- **A0** — anonymized IDs, no candidate pool, no features (the shuffle arm above).
- **A0.5** — anonymized IDs + the ML-ranked candidate pool, *no* features (control).
- **A1** — anonymized IDs + the pool + each candidate's structural feature vector, plus, each round, the mean feature profile of the revealed hits vs. non-hits, so the LLM can induce a decision rule and apply it.
- **A2** — real gene names (the C-arm).

Two design points matter. A1 is never shown the ML confidence score: were it shown, "reasoning" would collapse into copying the GBM. And **A0.5 exists because A1 is handed an ML-ranked pool that A0 is not** — without that control, A1 − A0 would confound "the features" with "being given a shortlist", and on the two smallest screens (K562-Essential: 623 genes; Scharenberg22: 1029) that shortlist is itself a strong prior.

Table 8: Reasoning vs. recall. Hit ratio at round 5, mean over **5 seeds**, same backend, $T = 0$. The three right-hand columns isolate one ingredient each, holding the others fixed, and are *seed-paired* (mean of per-seed differences) ± 1 s.d. — not differences of means. A2 reproduces its committed 5-seed values on all nine screens (max drift 0.025).

Screen	hit@R5 (mean) A0	A0.5	A1	A2	GBM	value of … (seed-paired ±s.d.) pool A0.5−A0	**features** A1−A0.5	name A2−A1
IFN-γ	0.180	0.160	0.175	0.190	0.183	−0.020 ±.003	+0.015 ±.008	+0.015 ±.015
IL-2	0.351	0.316	0.343	0.347	0.314	−0.035 ±.000	+0.027 ±.007	+0.004 ±.004
Sanchez21	0.088	0.090	0.090	0.095	0.087	+0.002 ±.002	+0.001 ±.005	+0.004 ±.005
Sanchez21 (dn)	0.091	0.093	0.099	0.095	0.101	+0.002 ±.002	+0.006 ±.001	−0.003 ±.005
Carnevale22	0.064	0.051	0.058	0.059	0.047	−0.013 ±.001	+0.007 ±.002	+0.001 ±.002
Scharenberg22	0.469	0.531	0.584	0.449	0.449	+0.061 ±.000	+0.053 ±.011	**−0.135** ±.011
Steinhart	0.075	0.048	0.065	0.145	0.090	−0.026 ±.003	+0.017 ±.008	**+0.080** ±.013
K562-Essential	0.286	0.318	0.597	0.549	0.476	+0.032 ±.043	**+0.279** ±.014	−0.048 ±.019
K562-GWPS	0.343	0.300	0.372	0.347	0.273	−0.043 ±.000	+0.072 ±.006	−0.024 ±.006
Average	0.216	0.212	**0.265**	0.253	0.224	−0.005	**+0.053**	−0.012

Three results, in order of how much they surprised us:

1. **The shortlist is worth nothing** (−0.005, negative on 5 of 9). A1's gain is not an artefact of being handed ML-curated candidates.
2. **The features are worth** +0.053**, and positive on all nine screens** (+0.001 to +0.279), every per-screen effect exceeding its seed s.d. — the most consistent effect anywhere in this paper.
3. **The gene name is worth** −0.012**.** With every gene renamed to a meaningless identifier and the structural features supplied, the system *matches or slightly exceeds* the real-name arm (0.265 vs. 0.253). On average, *the LLM does not need to know that a gene is IRF4; it needs the structure.*

Is this the LLM *reasoning*, or would any rule do? A1 hands the LLM the feature vectors and lets it induce a decision boundary. To test whether the *LLM* is what matters, we replace it with the most trivial boundary imaginable and change nothing else: score each candidate by its projection onto the (hit-centroid − non-hit-centroid) axis in the identical anonymized feature space, take the top B, no language model (round 1 falls back to the static ranking, exactly as A1 pads). This one-line rule scores **0.268** averaged over the nine screens — it *matches*, and marginally exceeds, A1's 0.265, screen by screen (K562-Essential 0.587 vs. 0.597, Scharenberg22 0.571 vs. 0.584, Steinhart 0.083 vs. 0.065, IL-2 0.351 vs. 0.343). So A1 is **not** evidence that the LLM reasons: the value is that the structural features are linearly separable, and a centroid rule extracts it as well as a language model does. What the decomposition establishes is therefore narrower — and cleaner — than "the LLM reasons": it is that *the gene name is dispensable once the structure is supplied*, with the LLM merely one (here interchangeable, and strictly unnecessary) way to map that structure to a choice.

That average, however, hides a bimodality that is the whole point:

- **Steinhart: the name is worth** $+0.080 \pm 0.013$, the largest anywhere, and the features recover almost nothing ($+0.017 \pm 0.008$; A1 at 0.065 is even *below* A0 at 0.075, and the two

error bars do not come close to overlapping). This is exactly the screen of Table 9: the one gain-of-function (CRISPRa) screen, where every DepMap feature is knockout-derived and does not transfer. The features we are able to hand the LLM are precisely the ones that are uninformative there, so nothing we supply substitutes for knowing what IRF4 does.

- **Scharenberg22: the name is worth** -0.135 ± 0.011 — it actively hurts, consistent with the shuffle *improving* that screen.

So substituting structure for the gene name is achievable and essentially free on eight of nine screens, and impossible on the one screen where biology is doing real work. We read this as a limit of what our feature table *contains*, not as proof that structure cannot suffice: our features are anchor-relative PPI / co-expression / pathway overlap plus constraint and essentiality, and none of them encodes the specific T-cell biology Steinhart rewards. The obvious reading of the bullet above — that Steinhart is special *because* it is the gain-of-function screen — is tempting, and wrong: we tested it directly (next), and perturbation modality does not explain it.

One further calibration: **A0 (0.216) scores below the plain GBM (0.224)**. An LLM branch with nothing to say does not merely fail to help — at a 0.4 fusion weight it drags the hybrid below the ML model it was meant to improve.

Testing the explanation: is it the gain-of-function modality? (No.)

The story above rests on a sample of one: Steinhart is the only gain-of-function (CRISPRa) screen in the benchmark, so "gain-of-function makes the gene name matter" is a hypothesis fitted to a single point. We tested it. The same study that supplied our IL-2 and IFN-γ screens (Schmidt et al., 2022) ran *both* a CRISPRi and a CRISPRa genome-wide screen for each cytokine, so the CRISPRa arm is the matched gain-of-function counterpart of two screens we already have. We onboarded `IL2_crispra` and `IFNG_crispra` from that arm, keeping everything fixed except the modality: the same anchor-relative features (they are modality-agnostic — activating vs. silencing IL-2 biology shares the same anchors), the same hit convention, an activation-framed task prompt ("genes that, when *over-expressed via CRISPRa*, regulate IL-2 production"), and — because these large screens would otherwise route to the LLM-suppressed `ml_heavy` regime — the same fusion weight as Steinhart ($w_{\text{LLM}} = 0.4$).

If gain-of-function were the cause, the gene name should be worth $\approx +0.08$ here too. It is not:

Screen (gain-of-function, CRISPRa)	name value (A2−A1)	cf. its CRISPRi twin
`IL2_crispra`	$+0.002 \pm 0.003$	-0.050
`IFNG_crispra`	-0.004 ± 0.001	$+0.013$
Steinhart (reference)	$+0.080 \pm 0.013$	—

On both new gain-of-function screens the gene name is worth nothing, an order of magnitude below Steinhart. **Perturbation modality does not explain Steinhart's name-dependence.** The corrected-prompt run barely moved these numbers from a first run with a placeholder prompt, so this is not an artefact of under-specifying the task; the name simply does not help here.

What *does* distinguish Steinhart is the shape of its hit set: 145 concentrated hits (0.8%) dominated by textbook exhaustion regulators (IRF4, TOX, ...) that a language model knows cold, against $\sim$900 diffuse top-movers ($\sim$5%) on each CRISPRa cytokine screen, where recognizable names are a small minority. Our loss-of-function-trained prior also transfers poorly to CRISPRa hits (hit@R5 ≈ 0.12 vs. 0.35 on the CRISPRi twin), so the whole system is weak there. We leave the

hit-set-concentration hypothesis untested. The operational consequence is clear enough, though: **we do not route by modality** — the structure-only reasoning mode stays an opt-in choice, not a default triggered by whether a screen is CRISPRa.

An accidental confirmation: the feature router already knew

The C-arm selects its feature set per screen. That router was tuned *empirically*, by trying feature sets and keeping whatever scored best, long before we understood why. It converged on excluding **all four DepMap features** for exactly two screens — Steinhart and K562-Essential — because including them *hurt*. The code comment recording this guessed at the reason ("DepMap disrupts online learning for GD2 biology"), and that guess was wrong: Steinhart does not measure GD2 biology at all (Table 1).

The real reason is the one above. **Every DepMap feature is derived from knockout screens**, and Steinhart is the one *gain-of-function* (CRISPRa) screen: it asks which genes make CAR-T cells resist exhaustion *when over-expressed*. A gene that is dispensable when deleted can be potent when over-expressed — IRF4, among Steinhart's strongest hits, is essential in only 6% of DepMap cell lines — so a knockout-derived prior is not merely uninformative there, it is actively misleading, and the router learned to discard it.

Reading the SHAP attributions back out confirms that this is what the model actually does:

Table 9: The ML model's most-used features, per screen (mean |SHAP| over the genes it selected). DepMap essentiality dominates wherever it is available — and is absent exactly where a knockout-derived prior does not transfer.

Screen	Perturbation	Top features used by the ML model
IFN-γ, IL-2, Carnevale22, Scharenberg22, K562-GWPS	loss-of-function	**depmap_frac_ess** (rank 1), then PPI / pLI
Steinhart	**gain-of-function** (CRISPRa)	PPI, STRING degree, pathway counts — **no DepMap feature at all**
K562-Essential	loss-of-function	PPI, STRING degree — **no DepMap feature**

This closes the loop on why Steinhart is the screen where the LLM matters most: it is precisely the screen where the ML model's single strongest feature family has been removed, leaving it with weak topological signal — and the LLM's T-cell biology is what fills the gap. (K562-Essential drops DepMap for a different reason: it *is* a curated essentiality screen, so a pan-cancer essentiality prior is close to a restatement of the label and adds nothing.)

A freely planning tool-using agent underperforms the constrained pipeline

The natural next step from a pipeline is a *tool-using agent* in the BioDiscoveryAgent / PerTurboAgent mould: let the LLM plan, call an ML-ranking tool and a pathway-enrichment tool, and commit a batch itself. We implemented exactly that (a task-specialised harness with a JSON action protocol; `ml_rank` / `enrich` / `finish`; no agent framework) and benchmarked it against the pipeline on identical screens and budgets. We state the scope up front: this is one action set, one prompt, one stopping rule and one budget, so the result below is evidence that *a freely planning*

agent does not beat the constrained fusion policy here, not that tools are useless in general — a different action schema or a policy that cannot override a confident ML score could close the gap.

Table 10: Tool-using agent vs. the deterministic pipeline (hit ratio @ R5). The agent plans its own rounds and calls tools; the pipeline fuses online ML with LLM reasoning under a fixed policy. The agent is *worse on 8 of 9 screens*.

Dataset	Tool-using agent	Pipeline (C)	Δ
Scharenberg22	**0.582**	0.465	+0.116
Carnevale22	0.044	**0.056**	−0.012
IFN-γ	0.163	**0.190**	−0.027
Sanchez21	0.059	**0.087**	−0.029
Sanchez21 (dn)	0.049	**0.099**	−0.050
Steinhart	0.100	**0.159**	−0.059
IL-2	0.264	**0.347**	−0.083
K562-GWPS	0.209	**0.347**	−0.138
K562-Essential	0.413	**0.556**	−0.143
Average	0.209	**0.256**	**−0.047**

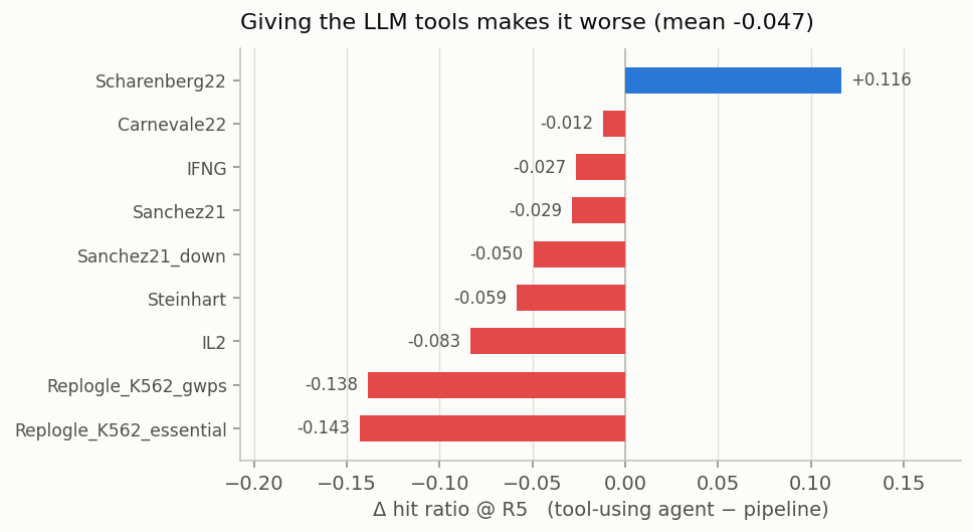

Figure 4: Per-screen delta of the tool-using agent against the pipeline. It wins on exactly one screen and loses worst precisely where the ML signal is strongest (K562-Essential, K562-GWPS) — given freedom, the LLM *overrides* a well-calibrated model.

The agent wins on exactly one screen (Scharenberg22, +0.116), where its runtime pathway-enrichment tool pays off, and loses most on the two genome-wide screens where the ML prior is strongest. The failure mode is instructive: given the freedom to act, the LLM overrides a model that is better calibrated than it is. **Constraining the LLM to a fixed fusion policy is not a limitation of the pipeline — it is the reason the pipeline wins.**

Transplanting the agent's one good idea

Since enrichment was the agent's only advantage, we transplanted it into the pipeline (Enrichr over the hits revealed so far, injected into the LLM prompt each round, gated to significant and coherent pathways), and separately tested an externalised *skill library* distilled from past experiments.

Table 11: Agent-style augmentations of the pipeline, as **paired** deltas over the same runs. Both are within noise of zero.

Augmentation	C	C + augmentation	Δ (paired)
Runtime pathway enrichment (gated)	0.260	0.263	+0.003
Externalised skill library	0.258	0.259	+0.001

Runtime enrichment captures the Scharenberg22 gain (+0.061 on that screen) without the collateral damage the free agent suffered, but nets only +0.003 overall: on genome-wide essentiality screens the enriched terms are highly significant yet *non-selective* (ribosome, proteasome, spliceosome), so "prioritise genes in these pathways" barely narrows a 9,000-gene pool. The skill library adds +0.001. Together with the memory ablation (+0.000), this is a consistent negative result about *externalisation*: memory, skills and retrieved pathways all add ≈ 0 once the model already has its parametric knowledge plus the hits it has observed.

Explainability: what is the LLM actually contributing?

The ablations say *how much* each component is worth; they cannot say *where* the value comes from. We therefore instrumented the arm to record, for every selected gene, a counterfactual attribution against the ML model's own top-k:

- **both** — the LLM named it *and* the ML would have selected it anyway;
- **LLM-only** — the LLM named it and the ML would *not* have selected it (this is the LLM's marginal contribution);
- **ML-only** — the LLM did not name it; it entered on ML score alone.

Only genes the LLM actually named count as LLM picks. The LLM branch pads its batch from the static ranker (Table 4†), and crediting that padding to the LLM would make "ML and LLM agree" partly mean "two ML models agree". An earlier version of this analysis made exactly that error: it reported an agreement bucket of $n = 800$ at a 21.1% hit rate, of which ≈87% was static-ranker padding. Excluding it sharpens the result considerably.

Pairing each pick with its revealed outcome gives the hit rate *per source* (9 screens × 5 rounds, ≈4,200 selected genes). K562-GWPS is reported separately: it is the one screen routed through the two-stage policy (the ML shortlists 384 candidates and the *LLM* makes the final selection), so the "ML-only" category cannot exist there and pooling it would corrupt the rates.

Table 12: Hit rate by attribution source, over the eight weighted-fusion screens. *Pooled* counts every selected gene once; *macro* averages the per-screen rates (screens with ≥ 10 picks in that category). Genuine agreement is rare — only 101 of ≈4,200 picks — but it is by far the strongest signal, at roughly **three times** the ML's own hit rate. The LLM's unilateral picks are *worse* than the ML's when pooled and about equal when macro-averaged, so the defensible claim is that they are *no better*.

Source	Genes selected	Hits	Hit rate (pooled)	Hit rate (macro)
ML + LLM agree	101	47	**46.5%** ± 9.7	**41.5%**
ML only	3,307	487	14.7% ± 1.2	14.9%
LLM only	752	61	8.1% ± 2.0	13.1%

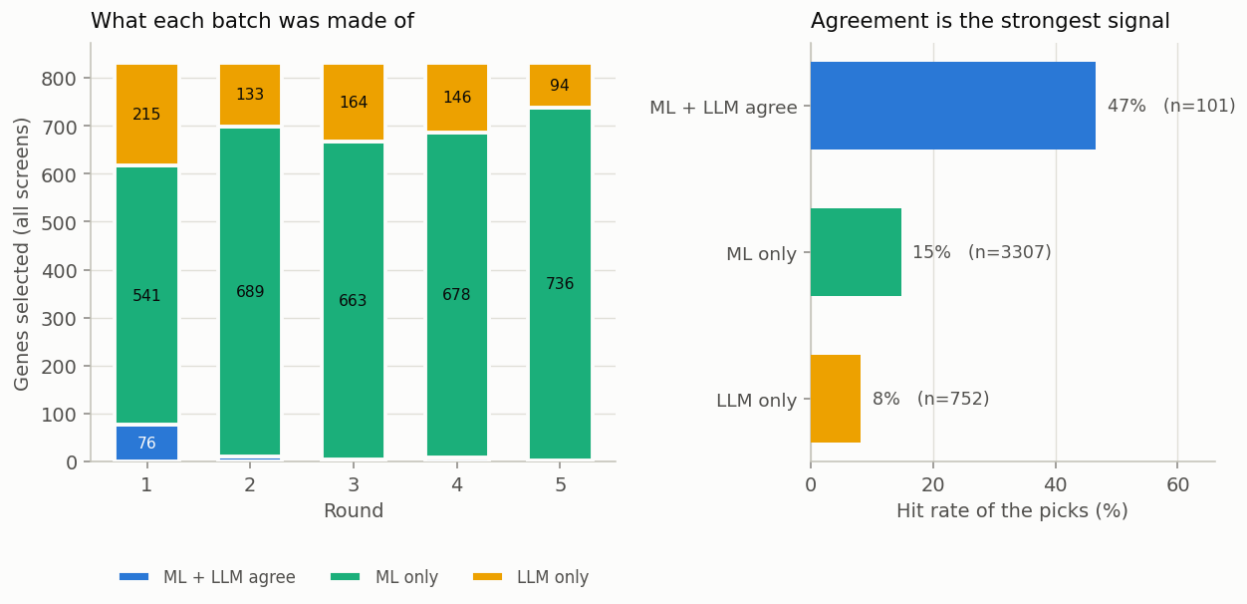

Figure 5: Left: what each round's batch is made of, pooled over the eight weighted-fusion screens. Right: the hit rate of each source. Genes that *both* components endorse hit at roughly twice the rate of genes the LLM introduces on its own.

The LLM is a verifier, not a proposer. The LLM is a *poor proposer*: left to itself it picks genes that hit at 8.1%, well below the ML's own 14.7%. But it is a *strong verifier*: when it independently names a gene the ML also ranks in its top-k, that gene hits at 46.5% — roughly three times the ML's rate. Such agreement is rare (101 of ≈4,200 picks, i.e. 2.4%), which is precisely why the component is worth so little on average and so much where it fires. This single measurement explains the two results above that otherwise look unrelated:

- why **removing the LLM costs only** 0.008 — the genes it proposes on its own are *worse* than what the ML would have picked instead, so losing them costs nothing; what is lost is a rare but highly informative agreement signal;
- why **giving the LLM tools makes it worse** (−0.047, Table 10) — a free agent selects mostly on its own judgement, which is precisely the weakest of the three sources.

The design implication is the opposite of the prevailing "more agency" direction: the LLM should be used to *re-weight* a calibrated model's candidates, not to choose freely.

Does endorsement add signal *beyond* the ML score? (Yes — but only where the ML is already confident.) The 46.5% agreement rate has an obvious confound: genes both components endorse might simply be the ones with the highest ML score, in which case the LLM adds nothing once that score is held fixed. We test this directly. On the same eight weighted screens, we bin every selected gene into ML-score deciles *within its screen* (so each screen's model scale cancels) and, within each bin, compare the hit rate of genes the LLM named against genes it did not. The value of naming is not constant — it is concentrated entirely in the top of the ML ranking:

Table 13: Hit rate by whether the LLM named the gene, *stratified by within-screen ML-score tertile* (eight weighted screens; the two-stage screen is excluded, its source labels being degenerate). LLM endorsement is worth nothing — slightly negative — in the bottom two tertiles, and worth 2.5× in the top tertile, where the ML is already confident. This is the verifier effect surviving a control for the ML score.

Within-screen ML tertile	LLM named (hit%, n)	not named (hit%, n)	ratio
bottom	7.7% (n=614)	11.3% (n=1050)	0.68×
middle	10.8% (n=130)	15.2% (n=1118)	0.71×
top	**43.1%** (n=109)	17.4% (n=1139)	**2.48**×

A logistic fit on the pooled genes puts a number on it: with the within-screen ML rank percentile and screen fixed effects in the model, LLM endorsement carries an independent odds ratio of **1.68** (screen-clustered bootstrap 95% CI $[1.05, 2.52]$, n = 4,160 genes over eight screens). The interval excludes 1, so endorsement predicts hits *after* adjusting for the ML score — the agreement effect is not merely a restatement of "high ML score." But the tertile split is the sharper statement: the LLM is a verifier of *confident* ML candidates specifically, adding nothing where the ML is unsure and confirming strongly where it is not. That is exactly the regime a fixed-fusion policy exploits and a free agent discards.

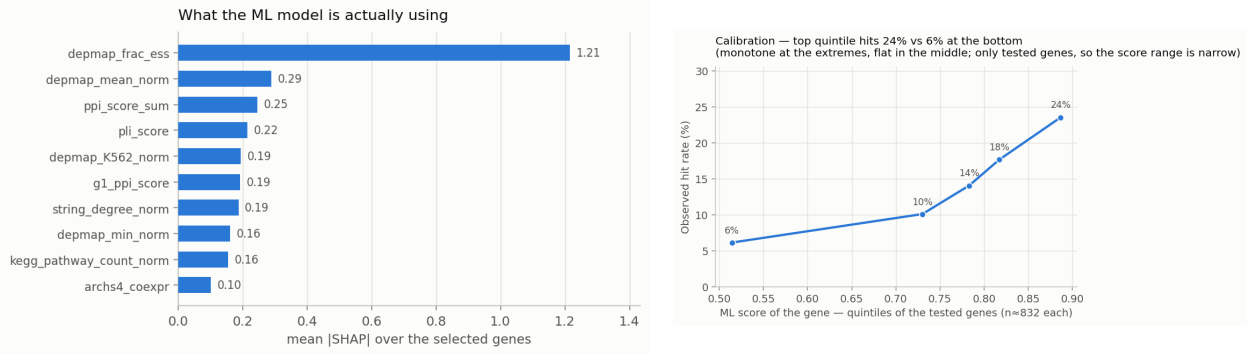

Figure 6: Left: features driving the ML score over the genes actually selected (LightGBM SHAP contributions; DepMap essentiality and PPI connectivity dominate). Right: calibration — the top-scoring quintile of tested genes hits about twice as often as the bottom quintile, though the curve is flat in the middle and the range is narrow because only *selected* genes are ever tested.

Limitations

We cannot separate the LLM's biological knowledge from memorisation. This is the most important caveat on every LLM result above. Seven of the nine screens are published, so their hit genes may sit in the model's training data, and a gene the model "knows" is a hit is indistinguishable — to our measurements — from a gene it reasons its way to.

It is tempting to appeal to the Steinhart screen as a clean control: BioDiscoveryAgent introduced it precisely because it was *unpublished*, stating that it is therefore "not part of the language model's training data". That argument was sound *for their models* (Claude 3.5 Sonnet, whose training cutoff precedes their release). **It does not transfer to ours.** BDA released the screen in their public repository (`ground_truth_Steinhart_crispra_GD2_D22.csv` and the corresponding `topmovers` file) in 2024, whereas our LLM is Claude Haiku 4.5, released in late 2025. We therefore cannot claim the screen is outside our model's training data, and we withdraw the otherwise attractive inference that "the LLM contributes most on the one screen it provably cannot have memorised".

We judge the practical leakage risk to be low — the hit set exists only as an 18,000-row table of effect sizes and a binary `.npy` array, not the kind of artefact language models reliably memorise — but low is not zero, and the strong claim is not available to us. Note also that the gene-name-shuffle ablation (-0.084 on Steinhart) does *not* settle this: shuffling gene names destroys memorised gene–hit associations and genuine biological knowledge equally, so it cannot distinguish them. Settling the question requires a screen released *after* the model's training cutoff; none of our nine qualifies.

Other limitations.

- **The cross-experiment prior is largely recurrence, not discovery.** As the prior-probe analysis shows, its gain comes substantially from same-study sibling screens (-0.066 on paired screens when excluded) and from generic knockout essentiality (DepMap-only $\approx$ the full model, and a trivial cross-screen hit-frequency count matches it). It is near-random on screens whose hits are mostly novel. It predicts what recurs; it does not find new phenotype-specific biology.
- **Not a verbatim reproduction of BioDiscoveryAgent.** We use their data and non-essential convention. We *did* run their agent as a controlled comparison (Table 2), but with our backend and model (Haiku 4.5) and small harness adaptations, not their exact published configuration; the controlled 0.105 is close to their reported 0.127 (Sonnet 3.5). Our headline gain over them is a cross-experiment prior they do not have, not a better agent.
- **The B baseline is padded**, on some screens heavily (86% on K562-Essential); it is not a pure-LLM baseline (Table 4†).
- **Genuine ML/LLM agreement is rare** ($n = 101$), so its hit rate carries a wide interval ($46.5\% \pm 9.7$). The *ordering* of the three sources is robust; the exact rate is not.
- **The attribution excludes K562-GWPS**, whose two-stage route lets the LLM make the final selection, so an "ML-only" pick cannot exist there by construction.
- **Calibration is measured only over genes the agent chose to test**, so the score range is narrow by construction and the curve is flat in its interior.
- **Nine screens, five seeds, no prospective wet-lab validation.** The arm is stochastic, which is why every component claim is reported as a *paired* delta.